Day 06

Production LLM API engineering

A model call is one line of code. A production LLM app is **everything around that line**: context, failures, speed, usage, cost and reliability.

1 Architecture · 2 Stateless vs stateful · 3 Context management · 4 Streaming (SSE) · 5 Errors · 6 Timeouts & retries · 7 Rate limits & quotas · 8 Tokens & cost · 9 Latency · 10 Models & fallback

1 · the shape of every AI app

Client → Backend → Model API

Client

- Chat UI: user types, sees the answer
- Calls `POST /chat` on **your** backend
- Holds **no secrets**

React · Next.js · mobile

→
←

Backend

- Validates input, checks auth
- Adds system prompt + context
- API key from `.env`
- Calls the model, handles errors, logs

FastAPI · Node

→
←

Model API

- Runs the hosted model
- Returns text + **usage** (tokens)
- An interface, not the model itself

OpenAI · Anthropic · Gemini

browser → model API directly	client → backend → model API
API key shipped to every visitor	Key stays on the server
No login, no per-user budget	Auth, quotas, budgets enforced
System prompt readable and editable	Prompt + business logic stay server-side
Nowhere to add RAG, tools, a DB, logs	The place where RAG, agents and tools live

live: asked "Explain RAG", the model said **"Red, Amber, Green"**. Course context in the system prompt fixed it.

2 · the model remembers nothing

Stateless vs stateful

Stateless

Each request is handled on its own, using only what is sent with it.

call 1: "My name is Sarthak."
call 2: "What is my name?"
→ "I can't tell your name unless you share it with me."

Stateful

The app keeps or fetches earlier turns and sends them along.

history = [user: My name is Sarthak,
assistant: Nice to meet you, user: What is my name?]
→ "Your name is Sarthak."

Where the state can live (the model API itself is always stateless):

Client	browser keeps the chat and sends it every time
Your backend + DB	load the conversation by `conversation_id` on each request (most common)
The provider	OpenAI `previous_response_id` chains calls. Call 2 still billed **43 input tokens**: old turns are re-read

remember: a stateless backend can still have a database. "Stateless" means no memory **between requests**, not no storage.

3 · choose what goes in the window

Conversation history & context management

Input tokens per turn, a 40-turn chat:

turn	full history	last 6 messages
1	45	45
10	1,033	332
20	2,129	332
40	4,321	332
total	87,280	12,629

History grows every turn, and you pay for all of it on every call. Too much also adds noise.

Window only · "What was my budget?" (stated in turn 1)
→ `"I'm unable to access your previous conversations..."`

Summary + window · same question, **426** tokens instead of 4,332
→ `"Your total budget for the trip to Japan is 2 lakh rupees."`

Sliding window
keep recent turns · forgets

Summarisation
compress older turns

Persistent memory
store useful facts · week 3

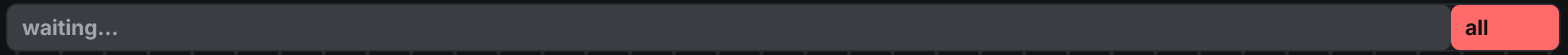
Retrieval
fetch only what's relevant · RAG, week 2

4 · feels faster, isn't faster

Streaming (SSE) vs non-streaming

Same prompt ("Explain RAG... about 150 words"), gpt-4o-mini, measured:

Non-streaming: nothing on screen until the end



blank for **2.10 s**, then 169 tokens at once

Streaming: words appear while the model writes



first words at **0.96 s**, finished at **2.27 s**, about the same total

```
event: delta
data: {"text": "R"}

event: delta
data: {"text": "AG"}

event: done
data: {"total_ms": ..., "output_tokens": ...}
```

- **SSE** = Server-Sent Events: one HTTP response that stays open; the server pushes `event:`/`data:` lines
- Browser side: `new EventSource(url)`
- The backend must **forward** chunks, not buffer the whole answer
- Errors mid-stream: the 200 is already sent, so send an `error` event

5 · classify, then react

Error handling & API failures

code	meaning	retry?
`400`	invalid request (e.g. `temperature=5`)	no, fix it
`401`	authentication failed (wrong key)	no, fix it
`403`	access forbidden	no
`404`	not found (model doesn't exist)	no
`429`	rate limit or quota	yes, back off
`500`	server error	yes
`503`	temporarily unavailable	yes
`504 / timeout`	took too long / network failed	yes

Triggered live: wrong key → 401 · fake model → 404 · temperature=5 → 400 · no server → APIConnectionError. Meanings can vary by provider.

Good error handling

- 1. Identify the error
- 2. Temporary or permanent?
- 3. Retry only when it can help
- 4. Show the user a helpful message
- 5. Log details (type, status, `request_id`), never secrets

user sees: "The AI service is temporarily unavailable. Please try again."
not: Traceback (most recent call last)...

6 · three different tools

Timeouts, retries & exponential backoff

Timeout

The longest you'll wait.

Retry

Try again after a **temporary** failure.

Backoff

Wait longer after each failure.

A fake API that fails 3 times with 503:

```
attempt 1 failed; waiting 0.47s
attempt 2 failed; waiting 0.96s
attempt 3 failed; waiting 1.74s
result: success after 4 calls
```

base 0.5 s × 2ⁿ, capped at 8 s, times random **jitter** so thousands of clients don't retry in sync.

OpenAI SDK default	value
timeout	`600 s (10 min!)`
max_retries	`2, with backoff`
retries on	`408 409 429 5xx, network`

```
OpenAI(timeout=20, max_retries=3)
```

rules: cap retries · respect `Retry-After` · never retry 400/401 · a timeout stops **your** waiting, the provider may still finish (and bill) the request.

7 · how fast vs how much

Rate limits & quota management

rate limit (speed)	quota (total)
60 requests / minute	₹1,000 daily budget
10,000 tokens / minute	1M tokens / month

Example values. Providers measure RPM, RPD, TPM, TPD and more.

The trap: 60 requests/min × 500 tokens = **30,000 TPM**. Within 60 RPM, but 3× over a 10,000 TPM limit → 429s.

Every response tells you where you stand:

```
x-ratelimit-limit-requests      30000
x-ratelimit-remaining-requests  29999
x-ratelimit-remaining-tokens    149999975
x-ratelimit-reset-tokens        0s
```

our account's real headers; yours depend on your tier

Protect your app:

Concurrency cap
asyncio.Semaphore(3): 10 calls, max 3 at once

Per-user budget
checked **before** the call: call 5 → [blocked]

Usage tracking
tokens per user, per day

Budget alerts
before the bill, not after

8 · log it on every call

Token usage & cost tracking

cost = **input tokens** / 1M × input price
+ **output tokens** / 1M × output price

Input = instructions + history + user message + retrieved docs. **Output** = the answer (plus hidden reasoning).

Example at $2 / $8 per 1M: 10,000 in = $0.020, 2,000 out = $0.016, **$0.036 per request**.

Log: model · tokens in/out · cost · latency · errors · `request_id`

measured	tokens	cost
gpt-4o-mini, 2-sentence answer	`18 in · 50 out`	`$0.000033`
… × 1 million requests		`$33`
gpt-5-mini, answer "No."	`96 out, 64 reasoning`	`$0.000197`
40-turn chat, full history	`87,280 in`	`$0.0131`
40-turn chat, last 6 messages	`12,629 in`	`$0.0019`

Prices per 1M tokens (checked 2026-10-16): gpt-4o-mini $0.15 in / $0.60 out · gpt-5-mini $0.25 / $2.00. Reasoning tokens are billed as output.

biggest levers: shorter history, smaller retrieved context, the cheapest model that passes your tests.

9 · what the user feels

Latency: TTFT vs total response time

TTFT

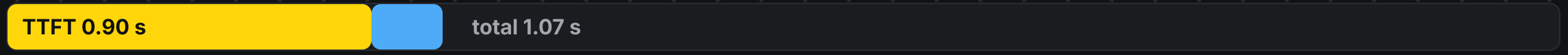
Time to first token: until the first chunk arrives. What the user **feels**.

Total time

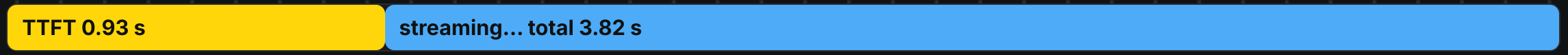
Until the whole answer has arrived. Grows with **answer length**.

Same model, streaming, measured:

short answer (30 tokens)



long answer (390 tokens)



10 runs of the same request, TTFT in seconds:

`0.86 0.93 0.95 0.99 1.08 1.10 1.37 1.50 1.50 1.70`

P50 1.08 s · **P95** 1.70 s (TTFT) · total P50 1.28 s · P95 2.80 s

P95 = 95% of requests finish within this time. The average hides the slow 5%, and those users leave.

With 10 samples, P95 ≈ the slowest run. In production, measure thousands.

10a · test on your task

Model selection

Quality	**Latency**	**Cost**	**Reliability**	**Capabilities**
solves your task?	fast enough?	at your traffic?	consistent?	vision, tools, JSON

6 support tickets → `billing` / `bug` / `account`, measured:

model	accuracy	avg latency	cost (6 calls)
`gpt-4o-mini`	6 / 6	1,337 ms	$0.000039
`gpt-5-mini`	6 / 6	1,080 ms	$0.000280 (7×)

Small model for simple jobs

Classification, routing, extraction. Same accuracy here at 1/7 the cost.

Bigger model for hard jobs

Multi-step reasoning, long answers, agents. Decide with **your** examples, not benchmarks.

10b · a plan for when it breaks

Provider fallback

Primary model
try the request
→
Failed?
classify the error
→
Temporary
retry, then backup model
/
Permanent
raise, alert, fix it

Primary times out (simulated outage):

```
primary gpt-5-mini failed (APITimeoutError),
  falling back
answered by backup gpt-4o-mini: RAG, or
  Retrieval-Augmented Generation, is...
```

Primary has a wrong key:

```
Raised, not hidden: AuthenticationError 401
```

Don't fall back blindly

- Bad credentials need fixing, not a backup
- The backup must support your **context length, tools, output format**
- And your **data rules** (where data may go)
- Often a different provider, via a router (OpenRouter, LiteLLM)

recap

Ten ideas, one sentence each

1 Architecture	Frontend → backend → model API. Keys stay on the backend.
2 State	Decide where conversation data is stored and retrieved. The model never remembers.
3 Context	Send the right information, not necessarily the entire history.
4 Streaming	Deliver chunks as they come; SSE is one common way.
5 Errors	Classify failures; respond safely; log without secrets.
6 Retries	Timeouts, bounded retries, exponential backoff with jitter.
7 Limits	Rate limits control speed; quotas control the total.
8 Cost	Track input/output tokens and spend per request.
9 Latency	TTFT = first output; total = completion. Watch P95.
10 Models	Optimise quality, speed, cost, reliability, with a sensible fallback.

try it yourself

The code for today

day-06/code/	shows
`01_architecture/`	FastAPI `/chat` backend + chat page + a Python client; mock model or real model; `leaky_client.html` anti-pattern (topic 1)
`02_streaming_sse/`	Same question, streaming vs non-streaming side by side in the browser, over SSE (topics 4, 9)
`day-06-production-llm-apis.ipynb`	Every number in these slides: state, context, streaming, errors, retries, rate-limit headers, budgets, cost, P50/P95, model choice, fallback (topics 2–10)

Homework: upgrade `01_architecture/main.py`: (1) keep history per `conversation_id` with a sliding window, (2) `timeout=20, max_retries=3`, (3) log tokens, cost and latency on every call, (4) fall back to a second model on a timeout or 5xx. Then kill your Wi-Fi mid-request and check the user sees a friendly message.

next: Day 07, evals & observability: see all of this per request, and know if the answers are good.

sources

Checked 2026-10-16

- OpenAI Responses API, text generation: developers.openai.com/api/docs/guides/text
- Conversation state (`previous_response_id`): developers.openai.com/api/docs/guides/conversation-state
- Streaming responses: developers.openai.com/api/docs/guides/streaming-responses
- Error codes: developers.openai.com/api/docs/guides/error-codes
- Rate limits (RPM/TPM, headers, backoff, Retry-After): developers.openai.com/api/docs/guides/rate-limits
- Model pages and prices: developers.openai.com/api/docs/models/gpt-4o-mini · /gpt-5-mini
- SDK defaults (timeout 600 s, max_retries 2, retry on 408/409/429/5xx): `openai` 3.31 Python SDK source
- Server-Sent Events: developer.mozilla.org/en-US/docs/Web/API/Server-sent_events/Using_server-sent_events

All measurements from `day-06/code`, run 2026-10-16 on gpt-4o-mini / gpt-5-mini. Latency and answers vary run to run.